Week_4

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t children[3];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    /* Create 3 child processes */
    for (int i = 0; i < 3; i++)
    {
        children[i] = fork();

        if (children[i] < 0)
        {
            perror("fork");
            exit(EXIT_FAILURE);
        }

        if (children[i] == 0)
        {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                   i + 1, getpid(), getppid());

            sleep(i + 1);

            printf("Child %d completed. PID = %d\n",
                   i + 1, getpid());

            /* Exit statuses: 10, 11, 12 */
            exit(10 + i);
```

```
    }
}

printf("All 3 child processes have been created.\n");

/*
 * wait() waits for any one child process.
 */
printf("\nUsing wait():\n");

pid_t finished_pid = wait(&status);

if (finished_pid > 0)
{
    if (WIFEXITED(status))
    {
        printf("wait() detected child PID %d finished "
               "with exit status %d\n",
               finished_pid, WEXITSTATUS(status));
    }
}

/*
 * waitpid() waits for specific child processes.
 */
printf("\nUsing waitpid():\n");

for (int i = 0; i < 3; i++)
{
    /*
     * The child already collected by wait() will
     * return immediately with an error if requested.
     */
    pid_t result = waitpid(children[i], &status, 0);
```

```
        if (result > 0)
        {
            if (WIFEXITED(status))
            {
                printf("waitpid() detected child PID %d "
                       "finished with exit status %d\n",
                       result, WEXITSTATUS(status));
            }
        }
    }

    printf("\nParent process completed.\n");

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~$ mkdir prac4
ramya@LAPTOP-OPDAF309:~$ cd prac4
ramya@LAPTOP-OPDAF309:~/prac4$ pwd
/home/ramya/prac4
ramya@LAPTOP-OPDAF309:~/prac4$ nano task1.c
ramya@LAPTOP-OPDAF309:~/prac4$ gcc task1.c -o task1
ramya@LAPTOP-OPDAF309:~/prac4$ ./task1
Parent Process Started
Parent PID: 1422
All 3 child processes have been created.

Child 1 started. PID = 1423, Parent PID = 1422
Using wait():
Child 2 started. PID = 1424, Parent PID = 1422
Child 3 started. PID = 1425, Parent PID = 1422
Child 1 completed. PID = 1423
wait() detected child PID 1423 finished with exit status 10

Using waitpid():
Child 2 completed. PID = 1424
waitpid() detected child PID 1424 finished with exit status 11
Child 3 completed. PID = 1425
waitpid() detected child PID 1425 finished with exit status 12

Parent process completed.
```

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid;

    printf("Parent process: PID = %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child PID = %d\n", getpid());

        printf("Child process terminating...\n");

        exit(0);
    }
    else
    {
        printf("Parent sleeping...\n");

        /*
         * Parent does not call wait().
         * Therefore, the terminated child becomes
         * a zombie process.
         */
```

```
        sleep(30);

        printf("Parent terminating...\n");
    }

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/prac4$ nano zombie.c
ramya@LAPTOP-OPDAF309:~/prac4$ gcc zombie.c -o zombie
ramya@LAPTOP-OPDAF309:~/prac4$ ./zombie
Parent process: PID = 1478
Parent sleeping...
Child process: PID = 1479
Child PID = 1479
Child process terminating...
Parent terminating...
```